Using NumPy for Linear Algebra

This is a short interactive tutorial that will give you some basic tools to do Linear Algebra operations. Embrace Linear Algebra, it is your friend!

Note that this is a runnable notebook and as such when you run a cell it stores the variables in memory. This is different to when you run a piece of code and when it ends they are destroyed. Thus you can sometimes run a cell out of order and issues can arise.

How to use this notebook: run the cells from top to bottom with Shift+Enter (or *Run → Run All Cells*). It runs to the end: the two deliberate error examples are caught and printed. **By the end you should have:** $5\mathbf{a}$, $\mathbf{a} \circ \mathbf{b}$ and $\mathbf{a} \cdot \mathbf{b} = 32$; $M\mathbf{a} = (14, 32)$; the inverse of $\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$; a singular matrix with determinant 0; and the solution $(x, y) = (1, 0)$ of a 2 × 2 system. You will use all of these in this week's tutorial.

When using NumPy we need to import the library

Click in the cell below and run the cell using Shift+Enter

In [3]:
```
import numpy as np
```

We can create vectors in NumPy by using the array data structure.

For example we can create the vectors $\mathbf{a} = \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix}$ and $\mathbf{b} = \begin{pmatrix} 4 \\ 5 \\ 6 \end{pmatrix}$.

```
a = np.array([1,2,3])
b = np.array([4,5,6])
```

We can also do the following multiplications.

Scalar Multiplication

e.g. $5\mathbf{a} = \begin{pmatrix} 5 \\ 10 \\ 15 \end{pmatrix}$

```
5 * a # [5,10,15]
```

Element-wise Multiplication (Hadamard Product)

This is not a common operation in Maths, but it is used a lot more than you might expect.

$$\mathbf{a} \circ \mathbf{b} = \begin{pmatrix} 1*4 \\ 2*5 \\ 3*6 \end{pmatrix} = \begin{pmatrix} 4 \\ 10 \\ 18 \end{pmatrix}$$

```
a*b # [1*4, 2*5, 3*6] = [4,10,18]
```

Dot Product

$$\mathbf{a} \cdot \mathbf{b} = 1 \cdot 4 + 2 \cdot 5 + 3 \cdot 6 = 32$$

```
np.dot(a,b)# 32
```

You can try this below, just hit Shift+Enter again.

In [ ]:
```
a = np.array([1,2,3])
b = np.array([4,5,6])

# scalar multiplication
print(f"Scalar multiplication of 5 * {a}")
print(5 * a)

# element-wise multiplication
print(f"\nElement-wise multiplication of {a} and {b}")
print(a * b)

# inner-product (dot product) multiplication i.e. row * vector
print(f"\nDot product multiplication of {a} and {b}")
print(np.dot(a,b))
```

A word on dimensions.

When creating a column vector of size 3 you are really creating an array. This is not strictly a column vector, it has dimensions (3,). It is missing an axis.

```
# This is not strictly a column vector, it has dimensions (3,). It is missing an axis.
a = np.array([1,2,3])
```

Most operations we do won't care about this, but I want you to see the full picture. Technically a mathematical column vector of size 3 has 3 rows and 1 column, its dimensions are 3×1 or in python `(3,1)`.

We can force the issue though!

```
# This is strictly a column vector, it has dimensions (3,1). It has two axes (three rows and 1 column)
b = np.array([
              [1],
              [2],
              [3]
             ])
```

Try these below. I have printed out the dimensions.

In [ ]:
```
# This is not strictly a column vector, it has dimensions (3,). It is mis
# You can think of (3,) as (3,1) most of the time
a = np.array([1,2,3])
print(f"The dimension of a is \n{a.shape}\n")

# This is strictly a column vector, it has dimensions (3,1). It has two a
b = np.array([
              [1],
              [2],
              [3]
             ])
print(f"The dimension of b is \n{b.shape}\n")
```

Matrices and Matrix Multiplication

We can also create matrices using the `np.array()` datatype.

To create a matrix with 2 rows and 3 columns

$$\mathbf{M_{2\times 3}} = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix}$$

use the following code.

```
# This is a matrix with 2 rows and 3 columns (2,3).
M = np.array([[1,2,3],
              [4,5,6]
              ])
```

We can also do matrix multiplication.

e.g.

$$\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix} \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix} = \begin{pmatrix} 14 \\ 32 \end{pmatrix}$$

Note that the number of columns of the first matrix must match the number of rows of the second matrix (in this case vector which is really just a 1 column matrix!).

```
# This is not strictly a column vector, it has dimensions (3,).
It is missing an axis.
a = np.array([1,2,3])
# This is a matrix with 2 rows and 3 columns (2,3).
M = np.array([[1,2,3],
              [4,5,6]
              ])

# We can do matrix multiplication because the inner dimensions of
M and a (2,3)(3,1) match!
answer = np.dot(M,a) # [14 32]
```

Here we use the `np.dot()` function. There are a couple of other ways, but we will stick to this one!

You can try this below. I have also demoed the other ways as well.

In [ ]:
```
# This is not strictly a column vector, it has dimensions (3,). It is mis
# You can think of (3,) as (3,1) most of the time
a = np.array([1,2,3])
print(f"The dimension of a is \n{a.shape}\n")

# This is strictly a column vector, it has dimensions (3,1). It has two a
b = np.array([
              [1],
              [2],
              [3]
             ])
print(f"The dimension of b is \n{b.shape}\n")

# This is a matrix with 2 rows and 3 columns (2,3).
M = np.array([[1,2,3],
              [4,5,6]
              ])

print(f"The dimension of M is \n{M.shape}\n")

# This is matrix multiplication with the .matmul function.
# It will work with either the non-strict or the strict column vector (a
# We can do matrix multiplication because the inner dimensions of M and a
# The same is true for M and b.
answer1 = np.matmul(M, a)
answer2 = np.matmul(M, b)
print(f"Output of .matmul() for M and a - \n{answer1}\n")
print(f"Output of .matmul() for M and b - \n{answer2}\n")

# This is matrix multiplication with the .dot function.
# It will work with either the non-strict or the strict column vector (a
answer3 = np.dot(M, a)
answer4 = np.dot(M, b)
print(f"Output of .dot() for M and a - \n{answer3}\n")
print(f"Output of .dot() for M and b - \n{answer4}\n")

# Later versions of Python now have an infix operator which does the same
# It secretly calls the .matmul() function.
answer5 = M @ a
answer6 = M @ b
print(f"M @ a - \n{answer5}\n")
print(f"M @ b - \n{answer6}\n")
```

Dimension Error

When the number of columns in the first matrix doesn't equal the number of rows in the second matrix we cannot do matrix multiplication.

e.g. The following matrix multiplication

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{pmatrix} \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix}$$

is not defined!

The number of columns of the first matrix (2) doesn't match the number of rows of the second matrix (3).

The code below shows that this results in a `ValueError` and it will let you know the issue!

In [ ]:
```
# This is not strictly a column vector, it has dimensions (3,). It is mis
a = np.array([1,2,3])

# This is a matrix with 3 rows and 2 columns (3,2).
M = np.array([[1,2],
              [3,4],
              [5,6]
              ])

# Now the dimensions of M and a do not match. You can think of (3,) as (3
# If we try and do matrix multiplication we will get an error.
try:
    print(M @ a)
except ValueError as err:   # expected: the dimensions do not match
    print('ValueError:', err)

# The error thrown here isn't that helpful, but it is saying that the dim
# (3,2)(3,1) Here the inner 2 and 3 do not match!
```

The same product with `np.dot()` instead of `@`. It fails too, but its message names both shapes, (3,2) and (3,), so the problem is easier to see.

In [ ]:
```
# This is not strictly a column vector, it has dimensions (3,). It is mis
a = np.array([1,2,3])

# This is a matrix with 3 rows and 2 columns (3,2).
M = np.array([[1,2],
              [3,4],
              [5,6]
              ])

# If we use np.dot instead we get a more useful error.
try:
    print(np.dot(M,a))
except ValueError as err:   # expected: the dimensions do not match
    print('ValueError:', err)

# Hence I always tend to use np.dot()
```

Matrix Inverse

We can compute the inverse of a matrix as follows.

$$\mathbf{A} = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$$

$$\mathbf{A}^{-1} = \frac{1}{ad-bc}\begin{pmatrix} d & -b \\ -c & a \end{pmatrix}$$

e.g.

$$\mathbf{A} = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$$

$$\mathbf{A}^{-1} = -\frac{1}{2}\begin{pmatrix} 4 & -2 \\ -3 & 1 \end{pmatrix} = \begin{pmatrix} -2 & 1 \\ \frac{3}{2} & -\frac{1}{2} \end{pmatrix}$$

We can do this in Python using the `np.linalg.inv()` function.

```
# This is a square matrix with 2 rows and 2 columns (2,2).
M = np.array([[1,2],
              [3,4]
              ])

# We can also find the inverse of a square matrix using
np.linalg.inv() (linalg is a linear algebra module)
print(f"The inverse of M is \n{np.linalg.inv(M)}")
```

Try the code below using Shift+Enter

In [ ]:
```
# This is a square matrix with 2 rows and 2 columns (2,2).
M = np.array([[1,2],
              [3,4]
              ])

# We can also find the inverse of a square matrix using np.linalg.inv() (
print(f"The inverse of M is \n{np.linalg.inv(M)}")
```

The following matrix is singular, this means it doesn't have an inverse.

$$\mathbf{A} = \begin{pmatrix} 1 & 2 \\ 3 & 6 \end{pmatrix}$$

Again using Python we can see this, in fact we get a `LinAlgError`. (In the browser version of this site NumPy returns a matrix of `nan`, "not a number", instead.)

```
# This is a square matrix with 2 rows and 2 columns (2,2).
M = np.array([[1,2],
              [3,6]
              ])

# The matrix above is singular, it does not have an inverse.
print(np.linalg.inv(M))

# It will throw a LinAlgError: Singular matrix error!
```

Try the code below using Shift+Enter

In [ ]:
```
# This is a square matrix with 2 rows and 2 columns (2,2).
M = np.array([[1,2],
              [3,6]
```

```
            ])

# The matrix above is singular, it does not have an inverse.
try:
    print(np.linalg.inv(M))   # in the browser (JupyterLite) NumPy prints
except np.linalg.LinAlgError as err:   # expected in Colab/Anaconda: Sing
    print('LinAlgError:', err)
```

The matrix above is singular because its determinant is $ad - bc = 1 \cdot 6 - 2 \cdot 3 = 0$, and the inverse formula has a $\frac{1}{ad-bc}$ in it. You can't divide by zero! The maths is telling you something!

Again Python can do this!

```
# This is because its determinant is 0.
# Guess what, there is a function for that too! Yay!
print(f"The determinant of M is {np.linalg.det(M)}")
```

Try the code below using Shift+Enter

In [ ]:
```
# This is because its determinant is 0.
# Guess what, there is a function for that too! Yay!
print(f"The determinant of M is {np.linalg.det(M)}")
```

Try it: what do you notice? In the singular-matrix cell above, change `M` to `[[1,2],[3,5]]` and run it again, then run the determinant cell.

Answer: the determinant is now $1 \cdot 5 - 2 \cdot 3 = -1$, which is not 0, so the inverse exists: $\begin{pmatrix} -5 & 2 \\ 3 & -1 \end{pmatrix}$. No error and no `nan` appear. A matrix has an inverse exactly when its determinant is not zero.

Solving Systems of Linear Equations

For the following system of linear equations.

$$\begin{aligned} x + 2y &= 1 \\ 3x + 4y &= 3 \end{aligned}$$

We can write it in matrix form.

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} 1 \\ 3 \end{pmatrix}$$

And solve using the matrix inverse.

$$\begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}^{-1} \begin{pmatrix} 1 \\ 3 \end{pmatrix}$$

$$\begin{pmatrix} x \\ y \end{pmatrix} = -\frac{1}{2} \begin{pmatrix} 4 & -2 \\ -3 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ 3 \end{pmatrix}$$

$$\begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} -2 & 1 \\ \frac{3}{2} & -\frac{1}{2} \end{pmatrix} \begin{pmatrix} 1 \\ 3 \end{pmatrix}$$

$$\begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$$

Spoiler, we can do this in Python!

```
# This is a square matrix with 2 rows and 2 columns (2,2).
A = np.array([[1,2],
              [3,4]
              ])

b = np.array([1,3])

# We can solve a system of linear equations using the matrix
inverse, np.linalg.inv().
x = np.dot(np.linalg.inv(A), b)
```

Try this below.

In [ ]:

```
# Here it is in python!

# This is a square matrix with 2 rows and 2 columns (2,2).
A = np.array([[1,2],
              [3,4]
              ])

b = np.array([1,3])

# We can solve a system of linear equations using the matrix inverse, np.
x = np.dot(np.linalg.inv(A), b)
print(f"The solution to the system of linear equations is\n {x}\n")

# On some computers the second value prints as a tiny number such as 2.2e
# Rounded, the answer is (1, 0).

print(f"Rounded to 2.dp, the solution to the system of linear equations i
```